

DOCUMENTO DE ARQUITECTURA
TÉCNICA · V0.1

SECURITY FRAMEWORK

Plano técnico completo de un framework de detección de amenazas en tiempo real: sensor ETW en Rust, motor de reglas en Go, esquema de eventos unificado, consola web y hoja de ruta para competir con Fibratus desde la experiencia ofensiva.

Ruby570bocadito · Diseño y arquitectura

Septiembre 2026 · nombre provisional del proyecto

Índice

1. Visión y Filosofía de Diseño	2
1.1 Defensa informada por ofensiva	2
1.2 Qué mejoramos respecto a Fibratus	2
2. Arquitectura General del Sistema	3
2.1 Flujo de datos extremo a extremo	4
2.2 Decisión de lenguaje: Go y Rust juntos	5
3. Componentes Principales	5
3.1 Sensor (Rust, Windows ETW)	5
3.2 Motor de detección (Go)	6
3.3 Consola web y API	6
3.4 Filaments v2 (extensiones Python)	6
4. Modelo de Datos y Esquema de Eventos	7
4.1 Pipeline de enriquecimiento	8
5. Sistema de Reglas YAML	9
5.1 Operadores soportados	9
5.2 Reglas sembradas desde TTPs ofensivas	10
6. Stack Tecnológico y Estructura del Repositorio	11
7. Roadmap de Desarrollo	12
8. Plan de Acción Inmediato	13
8.1 Ejecutar el tracer bullet	14
8.2 Primeros 30 días	14

1. Visión y Filosofía de Diseño

Este documento define la arquitectura técnica de un framework open source de detección de amenazas en tiempo real, concebido como alternativa moderna a Fibratus. El proyecto combina un sensor de eventos del sistema a nivel kernel con un motor de detección de comportamiento, escaneo de memoria con YARA y una consola web interactiva. El nombre del proyecto es provisional (security-framework) y se mantendrá hasta que la comunidad elija la marca definitiva. La versión descrita aquí corresponde a la v0.1: un núcleo funcional, honesto y verificable sobre el que crecer durante los próximos seis meses.

1.1 Defensa informada por ofensiva

La ventaja competitiva central del proyecto no es tecnológica sino cognitiva: quien diseña las detecciones ha operado del otro lado del tablero. Un red teamer sabe qué ruido produce Cobalt Strike al inyectar, cómo se comporta Mimikatz al abrir LSASS, y qué cadena de procesos deja tras de sí un phishing con PowerShell ofuscado. Ese conocimiento convierte las reglas de detección en algo cercano al comportamiento real de un adversario, no a patrones teóricos de laboratorio. Cada módulo del framework se plantea desde la pregunta: ¿cómo intentaré evadir esto en mi próxima operación, y cómo lo cierro?

De esta filosofía se derivan tres principios de diseño que gobiernan todas las decisiones de esta arquitectura. Primero, el comportamiento importa más que las firmas: las Indicators of Compromise caducan, las TTPs persisten. Segundo, la telemetría debe ser resistente a la evasión: el sensor se ancla en el kernel (ETW/eBPF) y no en hooks de user-mode que un malware pueda desinstalar. Tercero, la usabilidad es una feature de seguridad: una herramienta que nadie mira por complicada no detecta nada.

1.2 Qué mejoramos respecto a Fibratus

Fibratus es un excelente punto de referencia: su motor de reglas, su soporte ETW y su integración YARA demuestran madurez técnica. El análisis previo del proyecto identificó seis áreas donde este framework puede diferenciarse de forma clara. La tabla siguiente resume el posicionamiento objetivo, que el roadmap del capítulo 7 convierte en entregables concretos.

Área	Fibratus (estado actual)	Enfoque de este framework
Rendimiento	Bueno en cargas normales, con degradación bajo presión extrema de eventos.	Pipeline con buffers acotados, lotes y evaluación O(1) por regla indexada; presupuesto de latencia medido en CI.
Integración SIEM/SOAR	Exportación por formatos, sin conectores plug-and-play.	Conectores nativos para Elastic, Splunk y TheHive más API REST/gRPC documentada con OpenAPI.
Experiencia de usuario	Interfaz principalmente CLI.	Consola web React con gestión visual de reglas, dashboard en vivo y threat hunting interactivo.
Threat hunting	Detección reactiva basada en reglas conocidas.	Módulos de hunting con consultas predefinidas y YARA en memoria bajo demanda, diseñados desde TTPs ofensivas reales.
Contenedores y K8s	Soporte limitado en el núcleo.	Collector eBPF consciente de namespaces y cgroups, con empaquetado sidecar y DaemonSet.
Threat intelligence	Integrable pero periférico.	Motor de enriquecimiento central que consume MISP y STIX/TAXII y anota eventos en tiempo real.

Tabla 1. Comparativa de posicionamiento frente a Fibratus por área funcional.

< 10 ms latencia p99 de evaluación de reglas por evento	100 % cobertura proceso/fichero/red en el MVP	0 dependencias de driver kernel (ETW user-mode)
--	---	---

2. Arquitectura General del Sistema

La arquitectura sigue un modelo de tres planos desacoplados: recolección (sensor), análisis (motor) y consumo (interfaces). Cada plano escala y falla de forma independiente: si la consola web se cae, el motor sigue detectando; si el motor se reinicia, el sensor almacena en buffer y reintenta. Esta separación permite además distribuir los componentes: el sensor vive en cada endpoint, mientras el motor y las interfaces pueden centralizarse en un servidor de análisis. La Figura 1 muestra los cuatro estratos funcionales y el flujo de datos de arriba abajo.

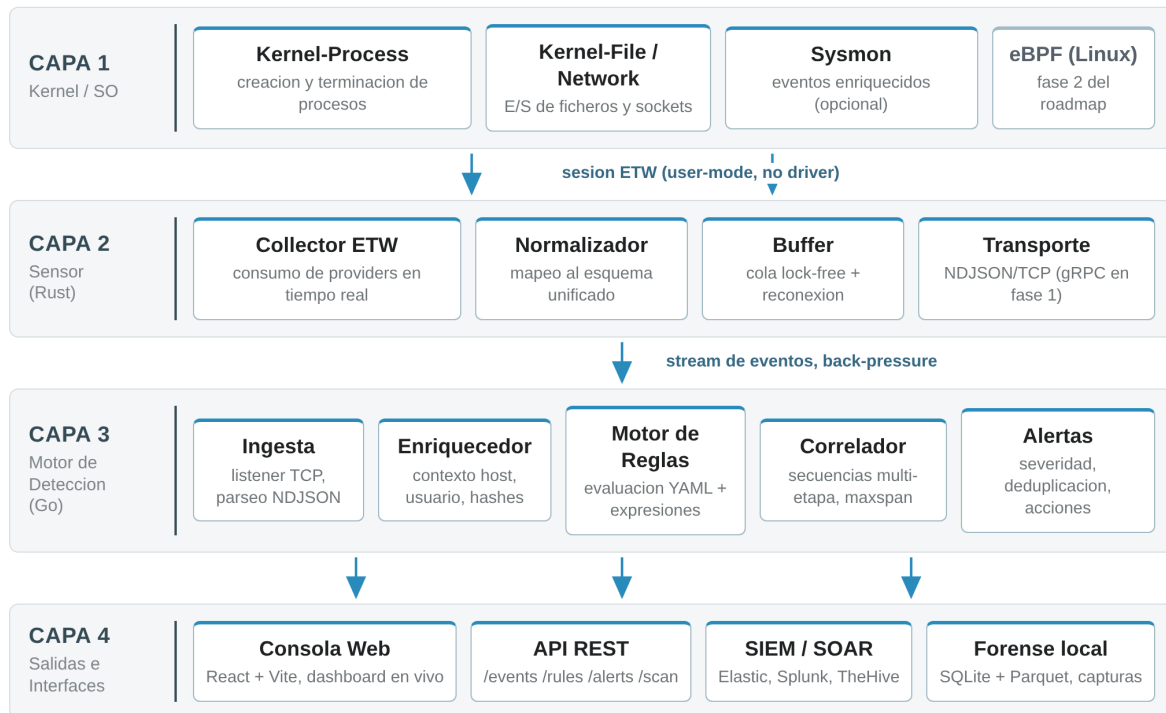


Figura 1. Arquitectura por capas del framework: kernel, sensor Rust, motor Go e interfaces de salida.

2.1 Flujo de datos extremo a extremo

El recorrido de un evento es siempre el mismo. El kernel emite notificaciones a través de los proveedores ETW de Windows (Kernel-Process, Kernel-File, Kernel-Network y opcionalmente Sysmon). El sensor Rust las consume en tiempo real mediante una sesión de trazado en user-mode, sin necesidad de driver propietario, y las traduce al esquema de eventos unificado del capítulo 4. Un buffer acotado absorbe los picos de telemetría y el transporte las envía por TCP en formato NDJSON al motor de detección.

En el motor, cada evento pasa por la ingesta (parseo y validación), el enriquecedor (contexto de host, usuario y reputación) y el motor de reglas, que evalúa las condiciones YAML indexadas por tipo de evento. El correlador mantiene una ventana temporal por host para detectar secuencias multi-etapa (por ejemplo, creación de proceso seguida de acceso a LSASS en menos de cinco segundos). Los hallazgos se convierten en alertas con severidad y acciones configurables, que se publican simultáneamente en la consola web, la API REST, los conectores SIEM y el almacén forense local.

2.2 Decisión de lenguaje: Go y Rust juntos

La combinación Go más Rust asigna a cada lenguaje el trabajo para el que es mejor. Rust protege el plano caliente del sensor: consume ETW a decenas de miles de eventos por segundo sin pausas de garbage collector y con garantías de seguridad de memoria en un componente que corre con privilegios altos. Go acelera el desarrollo del motor y sus interfaces: goroutines y channels modelan naturalmente el pipeline de eventos, y el ecosistema (gRPC, Gin, SQLite) reduce el tiempo hasta el MVP. El contrato entre ambos es el esquema de eventos y el protocolo de transporte, no el código compartido, lo que mantiene los equipos —y los binarios— independientes.

Componente	Lenguaje	Justificación técnica
Sensor / collector	Rust	Sin pausas de GC en el camino crítico de telemetría; ecosistema ferrisetw y windows-rs maduro para ETW; binario estático y pequeño para despliegue masivo en endpoints.
Motor de detección	Go	Concurrencia nativa (channels/select) para pipelines de eventos; compilación rápida; librerías de expresiones (expr-lang/CEL) y gRPC listas para producción.
Consola web	TypeScript	React 18 + Vite + Tailwind: ecosistema de componentes, hot reload y tipado estricto para un dashboard de datos en tiempo real.
Filaments (extensiones)	Python	Runtime embebido con sandbox para scripts de análisis; la comunidad de analistas vive en Python y el ecosistema de ciencia de datos importa.

Tabla 2. Asignación de lenguaje por componente y su justificación.

3. Componentes Principales

Este capítulo describe las responsabilidades internas de cada bloque de la Figura 1. El criterio de reparto es simple: un componente, una razón para cambiar. El sensor solo habla con el kernel y el transporte; el motor solo habla con eventos y reglas; las interfaces solo hablan con alertas y datos. Las interfaces Go entre módulos se definen en paquetes separados del código para que cada pieza sea sustituible y testeable por aislamiento.

3.1 Sensor (Rust, Windows ETW)

El sensor abre una sesión ETW por proveedor y consume los callbacks en hilos dedicados. Cada registro se mapea al esquema unificado sin enriquecimiento pesado: el sensor es rápido y tonto a propósito, porque cada microsegundo que gasta aquí es un microsegundo robado al endpoint monitorizado. Un buffer lock-free de capacidad fija absorbe picos; si un consumidor remoto se cae, el sensor aplica back-pressure, marca la pérdida y se reconecta con reintento exponencial. La telemetría de salud del propio sensor (eventos por segundo, profundidad de buffer, caídas) se publica junto a los eventos, porque un sensor ciego a sí mismo es un punto ciego del SOC.

3.2 Motor de detección (Go)

El motor es el cerebro del framework y se organiza en cinco etapas encadenadas por canales tipados. La ingesta acepta conexiones de múltiples sensores y valida cada evento contra el esquema. El enriquecedor añade contexto: usuario y host resoluciones, hashes de binarios, geolocalización de destinos de red y coincidencias contra feeds de inteligencia (MISP, STIX). El motor de reglas mantiene un índice por tipo de evento y evalúa solo las reglas candidatas, con presupuesto de latencia medible. El correlador construye secuencias con ventana temporal (maxspan) para ataques multi-etapa. El gestor de alertas deduplica, asigna severidad y dispara las acciones configuradas: log estructurado, notificación a la consola, webhook o acción de respuesta (matar proceso, aislar host) en fases posteriores.

3.3 Consola web y API

La consola web es la gran diferencia de experiencia frente a la CLI. Está construida como una SPA React que consume la API REST del motor y un stream WebSocket de alertas. Sus vistas mínimas para la v1 son cuatro: dashboard en vivo (alertas por severidad y línea de tiempo de eventos), gestor de reglas (editor YAML con validación y recarga en caliente), explorador de eventos con búsqueda por campos y módulo de hunting con consultas predefinidas. La API REST expone los mismos datos con OpenAPI generado desde el código, de modo que cualquier SOC pueda integrarse sin tocar la consola.

3.4 Filaments v2 (extensiones Python)

Los filaments heredan la idea de Fibratus de extensiones en Python, pero con dos mejoras de diseño: ejecución en un entorno aislado con recursos limitados y una API de host que expone el estado del motor (consultar eventos recientes, lanzar un escaneo YARA, pedir enriquecimiento) en lugar de solo callbacks. Cada filament declara sus triggers por tipo de evento y devuelve hallazgos con el mismo formato que las reglas nativas, de forma que la consola y los conectores no distinguen su origen. El sandbox ejecuta con timeout por script, sin acceso a red por defecto y con cuota de memoria, porque una extensión mal escrita no debe tumbar el detector.

Componente	Idioma	Responsabilidad principal	Contrato
Sensor	Rust	Consumir ETW, normalizar al esquema unificado y transmitir con back-pressure.	NDJSON/TCP
Ingesta	Go	Aceptar N sensores, validar y publicar al pipeline interno.	gRPC/REST (fase 1)
Enriquecedor	Go	Contexto de host, usuario, hashes y threat intelligence en línea.	interno (channel)
Motor de reglas	Go	Evaluar condiciones YAML indexadas por tipo de evento con presupuesto de latencia.	interno (channel)
Correlador	Go	Detectar secuencias multi-etapa con ventana temporal por host.	interno (channel)
Consola + API	TS/Go	Dashboard en vivo, gestión de reglas y hunting sobre eventos y alertas.	REST + WebSocket
Filaments v2	Python	Extender detección con scripts sandboxeados y API de host rica.	FFI/JSON bridge

Tabla 3. Responsabilidades y contratos por componente del framework.

4. Modelo de Datos y Esquema de Eventos

El esquema de eventos es el contrato maestro del sistema: lo emiten los sensores, lo validan las ingestas, lo indexan las reglas y lo consumen las interfaces. Se diseña inspirado en los estándares OCSF y ECS para que las integraciones SIEM sean casi gratuitas, pero reducido a los campos que la detección realmente usa. Cada evento es una línea JSON independiente, lo que hace el transporte trivial (NDJSON), el archivo forense lineal y el parseo paralelo sencillo.

Campo	Tipo	Descripción
id	uuid	Identificador único del evento generado por el sensor.
timestamp	RFC3339	Momento de alta resolución (nanosegundos) del evento en el kernel.
type	enum	process.create, process.terminate, file.write, network.connect, image.load, registry.set.
source	enum	etw, ebpf, sysmon, api: origen de la telemetría.
host	string	Hostname y dominio del endpoint monitorizado.
user	string	Usuario SID y nombre en el contexto del que se originó la actividad.
process.pid / ppid	int	Identificadores del proceso y su padre, base de los árboles de proceso.
process.name	string	Nombre del ejecutable (ej. powershell.exe), campo más consultado.
process.command_line	string	Línea de comandos completa, clave para detectar flags ofensivas.
process.image	path	Ruta absoluta del binario; alimenta el cálculo de hashes.

Campo	Tipo	Descripción
process.hashes	map	SHA-256 y MD5 del ejecutable para reputación y hunting.
network.*	objeto	L4/L7 del evento: protocolo, IP y puerto origen/destino, dominio resuelto.
file.*	objeto	Ruta, extensión, tamaño y hashes en eventos de fichero.
tags	list	Etiquetas libres: attack.t1059.001, hunting, filament:nombre.

Tabla 4. Campos del esquema de eventos unificado (v0.1).

EJEMPLO 1. EVENTO PROCESS.CREATE EN FORMATO NDJSON

```
{
  "id": "3f2a1c9e-8b4d-4f7a-9c21-7d0e5b6a1f88",
  "timestamp": "2026-09-29T14:22:31.4821193Z",
  "type": "process.create",
  "source": "etw",
  "host": "HR-WKS-042.corp.local",
  "user": "CORP\\jdoe",
  "process": {
    "pid": 6612,
    "ppid": 4820,
    "name": "powershell.exe",
    "command_line": "powershell.exe -nop -w hidden -enc SQBFAFgAIAAoAE4AZQB3...",
    "image": "C:\\Windows\\System32\\WindowsPowerShell\\v1.0\\powershell.exe",
    "hashes": {"sha256": "a3f1...c9d2"}
  },
  "tags": ["sensor:etw", "parent:winword.exe"]
}
```

4.1 Pipeline de enriquecimiento

El enriquecimiento ocurre después de la ingesta y antes de la evaluación de reglas, de modo que las reglas pueden condicionar sobre campos enriquecidos igual que sobre campos crudos. El pipeline aplica pasos idempotentes y configurables: resolución de usuario y host, cálculo de hashes con caché LRU por ruta, geolocalización y reputación de IPs externas, y contraste contra los feeds de inteligencia cargados (MISP, STIX/TAXII). Cada paso anota el evento sin mutar los campos originales del sensor, preservando la evidencia cruda para el forense. Los pasos costosos (feeds externos, geoIP) trabajan con caché y TTL, porque el enriquecimiento nunca puede ser el cuello de botella del pipeline de detección.

5. Sistema de Reglas YAML

El formato de reglas es la interfaz de usuario más importante del framework: es lo que un analista blue team escribe, lee y mantiene todos los días. Se elige YAML por legibilidad y compatibilidad con control de versiones, y se reserva la expresividad para un conjunto pequeño y predecible de operadores. Una regla declara a qué tipo de evento aplica, qué condiciones debe cumplir el evento, qué acciones disparar al detectar y qué etiquetas MITRE ATT&CK; lo caracterizan. Las reglas se cargan de un directorio, se validan al arrancar y se recargan en caliente al detectar cambios en disco, sin reiniciar el motor.

EJEMPLO 2. REGLA YAML DE DETECCIÓN DE POWERSHELL CODIFICADO

```
# rules/windows/powershell_encoded.yaml
name: "PowerShell con comando codificado"
id: 9f31c2a4-5d7b-4e18-8a02-3b9c6d1e7f40
description: >
  Detecta PowerShell con comandos Base64 (-enc/-EncodedCommand),
  patron habitual en phishing y execution ofuscada.
severity: high
event_type: process.create
conditions:
  - field: process.name
    operator: eq
    value: "powershell.exe"
  - field: process.command_line
    operator: contains_any
    value: ["-enc", "-EncodedCommand", "-w hidden"]
actions:
  - type: alert
    config:
      message: "PS con comando codificado en {host} por {user}"
      notify: true
tags: ["attack.t1059.001", "attack.execution"]
enabled: true
```

5.1 Operadores soportados

El conjunto de operadores de la v0.1 es deliberadamente minimalista: cada operador nuevo añade superficie de testeo y de evasión semántica. Con la familia siguiente se cubre la práctica totalidad de las detecciones orientadas a comportamiento, desde cadenas de procesos hasta acceso a rutas sensibles. Los operadores de secuencia pertenecen al correlador y no al evaluador puntual, y se describen en la misma tabla para referencia única.

Operador	Aplica a	Semántica y ejemplo de uso
eq	cualquier campo	Igualdad exacta. process.name eq "mimikatz.exe".
neq	cualquier campo	Desigualdad. process.parent.name neq "explorer.exe".
contains / contains_any	strings	Subcadena o lista de subcadenas (OR). command_line contains_any ["-enc", "-w hidden"].
startswith / endswith	strings	Prefijo o sufijo. image endswith "\\rundll32.exe".
regex	strings	Expresión regular RE2 validada al cargar la regla; se limita la longitud para evitar ReDoS.
in / not_in	listas	Pertenencia a lista estática o dinámica (feed TI). user in service_accounts.
gt / lt / between	numéricos	Comparación para puertos, tamaños o conteos. network.destination.port between [4444, 9999].
sequence + maxspan	correlación	Orden de eventos dentro de una ventana temporal por host: process.create de winword.exe seguida de network.connect externa en maxspan 10s.

Tabla 5. Operadores del motor de reglas y del correlador (v0.1).

5.2 Reglas sembradas desde TTPs ofensivas

El pack inicial de reglas no nace de catálogos genéricos: cada una corresponde a una técnica que el autor ha ejecutado o defendido en operaciones reales. Este origen ofensivo define el detalle de las condiciones, que apuntan a combinaciones de campos y no a nombres de binarios aislados, y por tanto resisten cambios triviales de nombre o firma. Las tres primeras reglas del repositorio cubren técnicas de la fase de ejecución y credential access, elegidas por su prevalencia en intrusiones reales.

- **T1059.001 - PowerShell encoded command:** powershell.exe con -enc, -EncodedCommand o -w hidden en línea de comandos; el falso positivo típico (software de gestión) se controla con una exclusión por firmante.
- **T1105 - Descarga con binarios lolbas:** certutil.exe con -urlcache o bitsadmin.exe con /transfer hacia destinos externos, correlado con process.create padre no explorador.
- **T1003.001 - LSASS dump vía comsvcs.dll:** rundll32.exe cargando comsvcs.dll con MiniDump en línea de comandos, o cualquier proceso con handle de escritura hacia lsass.exe (fase 1, requiere eventos de handle).

Cada regla del pack documenta en su descripción cómo fue validada: qué TTP se ejecutó en laboratorio, qué eventos generó y qué ruido produce en un host limpio. Esta disciplina de evidencia, heredada de la práctica ofensiva, es lo que permite que la comunidad confíe y contribuya packs completos por técnica MITRE.

6. Stack Tecnológico y Estructura del Repositorio

El stack se elige para minimizar fricción en las primeras semanas y para escalar después sin reescrituras. Se priorizan librerías mantenidas, con licencia permisiva y trazabilidad de rendimiento publicada. La tabla siguiente fija la decisión técnica por capa del sistema; el árbol del repositorio muestra dónde vive cada pieza y cómo se organiza el monorepo que alinea sensor, motor y consola bajo una única versión.

Capa	Tecnología	Motivo de la elección
Sensor (Rust)	tokio, ferrisetw, windows-rs, serde, anyhow	Runtime asíncrono estable; ferrisetw simplifica las sesiones ETW; windows-rs cubre las APIs nativas con bindings seguros.
Motor (Go)	stdlib net, expr-lang/expr, gopkg.in/yaml.v3, go-sqlite3, gin-gonic/gin	Listener concurrente con net; expresiones evaluadas con expr; YAML estándar; SQLite embebido para forense; Gin para la API REST.
Transporte	NDJSON sobre TCP (v0.1) → gRPC + protobuf (fase 1)	NDJSON permite depurar con netcat desde el día uno; gRPC aporta tipado fuerte, streaming y compresión al escalar a muchos sensores.
Consola web	React 18, TypeScript, Vite, Tailwind CSS, Zustand	Tooling instantáneo con Vite; Tailwind para consistencia visual; Zustand como estado ligero para streams en vivo.
Almacenamiento	SQLite (forense local), Parquet (archivos), S3/MinIO	SQLite sin instalación es ideal para evidencia en endpoint; Parquet para análisis columnar posterior; S3 para retención barata.
Build y CI	Make, GitHub Actions, Docker, goreleaser, cargo-dist	Un comando para compilar todo; releases reproducibles multi-plataforma y contenedores para entornos mixtos.

Tabla 6. Stack tecnológico por capa con su justificación.

EJEMPLO 3. ESTRUCTURA DEL MONOREPO DEL SCAFFOLD INICIAL

```

security-framework/
├── cmd/
│   ├── engine/           # binario del motor de detección (Go)
│   └── devsensor/        # sensor simulado para desarrollo (Go)
├── internal/
│   ├── ingest/           # listener TCP NDJSON + validación de esquema
│   ├── enrich/           # pipeline de enriquecimiento
│   ├── rules/            # parser YAML, índice y evaluador de reglas
│   ├── correlate/        # secuencias multi-etapa (fase 1)
│   └── alert/            # alertas, deduplicación y acciones
├── pkg/
│   └── model/            # esquema de eventos compartido (Go)
├── sensor/               # sensor ETW en Rust
│   ├── src/
│   │   ├── collector/    # sesiones ETW por proveedor
│   │   ├── normalize/    # mapeo al esquema unificado
│   │   └── transport/    # buffer + envío NDJSON
│   └── Cargo.toml
├── rules/                # reglas YAML (windows/, linux/)
├── web/                  # consola React (fase 3)
├── filaments/            # extensiones Python (fase 4)
├── docs/                 # este documento + ADRs
├── Makefile
├── Dockerfile
└── README.md

```

Las convenciones de código se fijan desde el primer commit: formato obligatorio (gofmt y rustfmt), lint en CI (golanci-lint y clippy), commits convencionales y ADRs numerados en docs/ para cada decisión arquitectónica relevante. Todo módulo recibe test unitario antes de fusionarse, y el pipeline integra una batería de eventos sintéticos que ejercita las reglas del pack inicial para detectar regresiones de detección, no solo de compilación.

7. Roadmap de Desarrollo

El roadmap se organiza en cuatro fases de seis a ocho semanas, cada una con un entregable demostrable al cierre. La regla de priorización es explícita: primero el flujo vertical que detecta (tracer bullet), después la anchura de telemetría, después la experiencia, y solo al final la extensibilidad. Cada fase concluye con un release etiquetado, notas de versión y una demo grabada, porque un framework open source que no publica no construye comunidad.

Fase	Ventana	Objetivos	Entregables de cierre
1. Core y MVP	Semanas 1-6 (Q4 2026)	Tracer bullet end-to-end; sensor ETW básico; motor de reglas YAML con operadores puntuales; CLI de prueba.	v0.1: pipeline proceso-regla-alerta funcionando; 10 reglas de ejemplo; binarios publicados en GitHub Releases.
2. Detección avanzada	Semanas 7-14 (Q1 2027)	Integración YARA para escaneo de memoria bajo demanda; collector eBPF para Linux; correlación de secuencias con maxspan; almacenamiento forense SQLite.	v0.2: detección multi-plataforma, correlador activo y capturas consultables desde la CLI.
3. Experiencia	Semanas 15-20 (Q1-Q2 2027)	Consola web con dashboard en vivo; editor de reglas con validación; API REST documentada; conectores Elastic y Splunk.	v0.3: consola desplegada con docker compose; OpenAPI publicado; conectores en paquete oficial.
4. Extensibilidad	Semanas 21-26 (Q2 2027)	Filaments v2 en sandbox Python; plugins de alto rendimiento; profiling y optimización de throughput; documentación completa.	v1.0: API de filaments estable, benchmarks publicados y guía de contribución activa.

Tabla 7. Fases del roadmap con ventanas temporales y entregables.

Las métricas de éxito miden la salud del proyecto, no solo el código. Para la fase 1: pipeline estable durante 24 horas continuas en un host de laboratorio con carga sintética, latencia p99 de evaluación bajo 10 milisegundos y las 10 reglas semilla disparando contra sus TTPs de prueba. Para las fases 2 y 3: cobertura de eventos de proceso, fichero y red por encima del 95 por ciento en Windows, tiempo medio de instalación bajo cinco minutos y primeros usuarios externos emitiendo feedback en el tracker. La estrategia de comunidad acompaña al producto desde el día uno: documentación de arranque en diez minutos, casos de uso por técnica MITRE publicables y una etiqueta good-first-issue siempre abierta.

8. Plan de Acción Inmediato

El scaffold incluido junto a este documento implementa el tracer bullet: un disparo vertical completo que valida la arquitectura antes de construir sobre ella. Su valor no es funcional sino de riesgo: demuestra que el esquema de eventos, el protocolo de transporte, el motor de reglas y las alertas encajan, y lo hace hoy, no en seis semanas. La Figura 2 muestra el flujo de los cinco pasos que ejecuta el scaffold en cualquier máquina con Go instalado, sin depender de Windows.

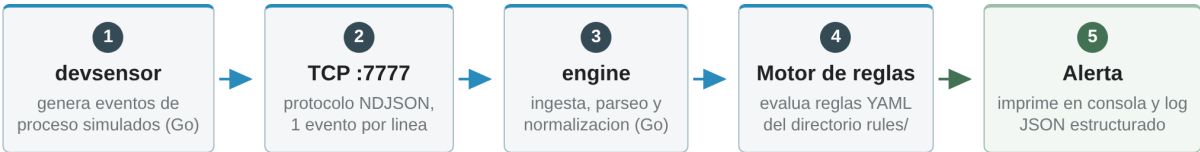


Figura 2. Tracer bullet del scaffold: pipeline simulado de evento a alerta ejecutable en cualquier plataforma.

8.1 Ejecutar el tracer bullet

El motor arranca en una terminal y escucha por TCP en el puerto 7777; el sensor de desarrollo genera en otra terminal eventos de proceso simulados, algunos benignos y otros emulando PowerShell codificado o descargas con certutil. En segundos la consola muestra las alertas disparadas con severidad y los eventos crudos en formato JSON estructurado. El mismo flujo, cambiando el devsensor por el binario Rust en un host Windows, consume ETW real: el motor no sabe la diferencia porque el contrato es el esquema de eventos.

EJEMPLO 4. COMANDOS DEL TRACER BULLET INCLUIDO EN EL SCAFFOLD

```
# 1. Arrancar el motor de detección (terminal 1)
make run-engine
# -> engine escuchando en :7777, reglas cargadas de rules/

# 2. Generar eventos simulados (terminal 2)
make run-devsensor
# -> envia eventos benignos y ofensivos al motor

# 3. Resultado esperado en la terminal del motor
# [ALERT] HIGH T1059.001 powershell.exe -enc ... host=LAB-WKS-01
# [EVENT] process.create notepad.exe pid=4212

# Compilar el sensor ETW para Windows (requiere toolchain Rust)
make build-sensor-windows
# -> target/x86_64-pc-windows-msvc/release/security-sensor.exe
```

8.2 Primeros 30 días

Con el tracer bullet validado, el siguiente mes convierte el andamiaje en producto mínimo. La lista siguiente ordena el trabajo por dependencia técnica: primero estabilizar el contrato de datos, después sustituir la simulación por telemetría real, y por último abrir el proyecto al mundo.

- **Semana 1:** congelar el esquema de eventos v0.1 con test de compatibilidad; publicar el repo con README bilingüe, licencia Apache 2.0 y plantillas de issue.
- **Semana 2:** implementar el collector ETW real sobre ferrisetw en el sensor Rust; probar contra los proveedores Kernel-Process y Kernel-File en un VM de Windows 11.
- **Semana 3:** ampliar el motor con recarga en caliente de reglas y el índice por tipo de evento; medir latencia p99 con carga sintética de 1.000 eventos por segundo.
- **Semana 4:** escribir los ADRs de transporte (NDJSON → gRPC), publicar la demo grabada del tracer bullet en el README y abrir la primera milestone v0.1 con issues etiquetadas.

A partir de ahí, el roadmap del capítulo 7 gobierna. El scaffold está preparado para crecer sin movimientos traumáticos: el transporte evoluciona a gRPC manteniendo el esquema, el correlador se acopla al pipeline como una etapa más, y la consola web consume la API que el motor ya expone. La arquitectura descrita en este documento es, sobre todo, una promesa de estabilidad para quien construya encima: el comportamiento del pipeline no cambiará, solo mejorará su implementación.